

8.3 Implementing Trees

The `AbstractTree` and `AbstractBinaryTree` classes that we have defined thus far in this chapter are both *abstract base classes*. Although they provide a great deal of support, neither of them can be directly instantiated. We have not yet defined key implementation details for how a tree will be represented internally, and how we can effectively navigate between parents and children.

There are several choices for the internal representation of trees. We describe the most common representations in this section. We begin with the case of a *binary tree*, since its shape is more strictly defined.

8.3.1 Linked Structure for Binary Trees

A natural way to realize a binary tree T is to use a *linked structure*, with a node (see Figure 8.9a) that maintains references to the element stored at a position p and to the nodes associated with the children and parent of p . If p is the root of T , then the parent field of p is null. Likewise, if p does not have a left child (respectively, right child), the associated field is null. The tree itself maintains an instance variable storing a reference to the root node (if any), and a variable, called *size*, that represents the overall number of nodes of T . We show such a linked structure representation of a binary tree in Figure 8.9b.

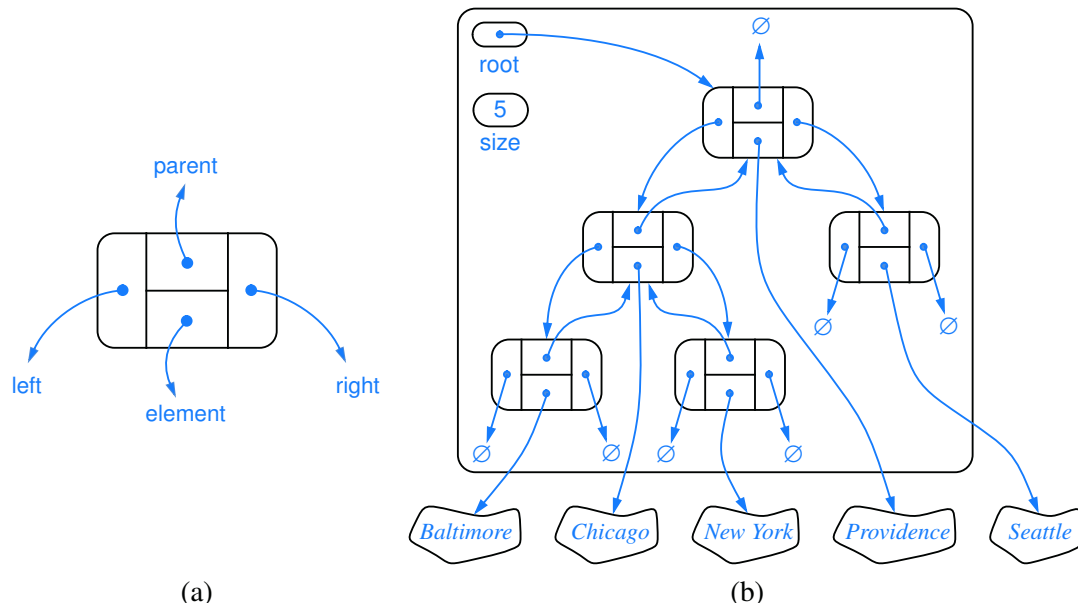


Figure 8.9: A linked structure for representing: (a) a single node; (b) a binary tree.

Operations for Updating a Linked Binary Tree

The `Tree` and `BinaryTree` interfaces define a variety of methods for inspecting an existing tree, yet they do not declare any update methods. Presuming that a newly constructed tree is empty, we would like to have means for changing the structure of content of a tree.

Although the principle of encapsulation suggests that the outward behaviors of an abstract data type need not depend on the internal representation, the *efficiency* of the operations depends greatly upon the representation. We therefore prefer to have each concrete implementation of a tree class support the most suitable behaviors for updating a tree. In the case of a linked binary tree, we suggest that the following update methods be supported:

- `addRoot(e)`: Creates a root for an empty tree, storing *e* as the element, and returns the position of that root; an error occurs if the tree is not empty.
- `addLeft(p, e)`: Creates a left child of position *p*, storing element *e*, and returns the position of the new node; an error occurs if *p* already has a left child.
- `addRight(p, e)`: Creates a right child of position *p*, storing element *e*, and returns the position of the new node; an error occurs if *p* already has a right child.
- `set(p, e)`: Replaces the element stored at position *p* with element *e*, and returns the previously stored element.
- `attach(p, T1, T2)`: Attaches the internal structure of trees *T*₁ and *T*₂ as the respective left and right subtrees of leaf position *p* and resets *T*₁ and *T*₂ to empty trees; an error condition occurs if *p* is not a leaf.
- `remove(p)`: Removes the node at position *p*, replacing it with its child (if any), and returns the element that had been stored at *p*; an error occurs if *p* has two children.

We have specifically chosen this collection of operations because each can be implemented in $O(1)$ worst-case time with our linked representation. The most complex of these are `attach` and `remove`, due to the case analyses involving the various parent-child relationships and boundary conditions, yet there remains only a constant number of operations to perform. (The implementation of both methods could be greatly simplified if we used a tree representation with a sentinel node, akin to our treatment of positional lists; see Exercise C-8.38.)

Java Implementation of a Linked Binary Tree Structure

We now present a concrete implementation of a `LinkedBinaryTree` class that implements the binary tree ADT, and supports the update methods described on the previous page. The new class formally extends the `AbstractBinaryTree` base class, inheriting several concrete implementations of methods from that class (as well as the formal designation that it implements the `BinaryTree` interface).

The low-level details of our linked tree implementation are reminiscent of techniques used when implementing the `LinkedPositionalList` class in Section 7.3.3. We define a nonpublic nested `Node` class to represent a node, and to serve as a `Position` for the public interface. As was portrayed in Figure 8.9, a node maintains a reference to an element, as well as references to its parent, its left child, and its right child (any of which might be null). The tree instance maintains a reference to the root node (possibly null), and a count of the number of nodes in the tree.

We also provide a `validate` utility that is called anytime a `Position` is received as a parameter, to ensure that it is a valid node. In the case of a linked tree, we adopt a convention in which we set a node's parent pointer to itself when it is removed from a tree, so that we can later recognize it as an invalid position.

The entire `LinkedBinaryTree` class is presented in Code Fragments 8.8–8.11. We provide the following guide to that code:

- Code Fragment 8.8 contains the definition of the nested `Node` class, which implements the `Position` interface. It also defines a method, `createNode`, that returns a new node instance. Such a design uses what is known as the **factory method pattern**, allowing us to later subclass our tree in order to use a specialized node type. (See Section 11.2.1.) Code Fragment 8.8 concludes with the declaration of the instance variables of the outer `LinkedBinaryTree` class and its constructor.
- Code Fragment 8.9 includes the protected `validate(p)` method, followed by the accessors `size`, `root`, `left`, and `right`. We note that all other methods of the `Tree` and `BinaryTree` interfaces are derived from these four concrete methods, via the `AbstractTree` and `AbstractBinaryTree` base classes.
- Code Fragments 8.10 and 8.11 provide the six update methods for a linked binary tree, as described on the preceding page. We note that the three methods—`addRoot`, `addLeft`, and `addRight`—each rely on use of the factory method, `createNode`, to produce a new node instance.

The `remove` method, given at the end of Code Fragment 8.11, intentionally sets the parent field of a deleted node to refer to itself, in accordance with our conventional representation of a defunct node (as detected within the `validate` method). It resets all other fields to null, to aid in garbage collection.

```

1  /** Concrete implementation of a binary tree using a node-based, linked structure. */
2  public class LinkedBinaryTree<E> extends AbstractBinaryTree<E> {
3
4      //----- nested Node class -----
5      protected static class Node<E> implements Position<E> {
6          private E element;                // an element stored at this node
7          private Node<E> parent;            // a reference to the parent node (if any)
8          private Node<E> left;              // a reference to the left child (if any)
9          private Node<E> right;             // a reference to the right child (if any)
10         /** Constructs a node with the given element and neighbors. */
11         public Node(E e, Node<E> above, Node<E> leftChild, Node<E> rightChild) {
12             element = e;
13             parent = above;
14             left = leftChild;
15             right = rightChild;
16         }
17         // accessor methods
18         public E getElement() { return element; }
19         public Node<E> getParent() { return parent; }
20         public Node<E> getLeft() { return left; }
21         public Node<E> getRight() { return right; }
22         // update methods
23         public void setElement(E e) { element = e; }
24         public void setParent(Node<E> parentNode) { parent = parentNode; }
25         public void setLeft(Node<E> leftChild) { left = leftChild; }
26         public void setRight(Node<E> rightChild) { right = rightChild; }
27     } //----- end of nested Node class -----
28
29     /** Factory function to create a new node storing element e. */
30     protected Node<E> createNode(E e, Node<E> parent,
31                                   Node<E> left, Node<E> right) {
32         return new Node<E>(e, parent, left, right);
33     }
34
35     // LinkedBinaryTree instance variables
36     protected Node<E> root = null;        // root of the tree
37     private int size = 0;                  // number of nodes in the tree
38
39     // constructor
40     public LinkedBinaryTree() { }          // constructs an empty binary tree

```

Code Fragment 8.8: An implementation of the `LinkedBinaryTree` class.
(Continues in Code Fragments 8.9–8.11.)

```

41 // nonpublic utility
42 /** Validates the position and returns it as a node. */
43 protected Node<E> validate(Position<E> p) throws IllegalArgumentException {
44     if (!(p instanceof Node))
45         throw new IllegalArgumentException("Not valid position type");
46     Node<E> node = (Node<E>) p; // safe cast
47     if (node.getParent() == node) // our convention for defunct node
48         throw new IllegalArgumentException("p is no longer in the tree");
49     return node;
50 }
51
52 // accessor methods (not already implemented in AbstractBinaryTree)
53 /** Returns the number of nodes in the tree. */
54 public int size() {
55     return size;
56 }
57
58 /** Returns the root Position of the tree (or null if tree is empty). */
59 public Position<E> root() {
60     return root;
61 }
62
63 /** Returns the Position of p's parent (or null if p is root). */
64 public Position<E> parent(Position<E> p) throws IllegalArgumentException {
65     Node<E> node = validate(p);
66     return node.getParent();
67 }
68
69 /** Returns the Position of p's left child (or null if no child exists). */
70 public Position<E> left(Position<E> p) throws IllegalArgumentException {
71     Node<E> node = validate(p);
72     return node.getLeft();
73 }
74
75 /** Returns the Position of p's right child (or null if no child exists). */
76 public Position<E> right(Position<E> p) throws IllegalArgumentException {
77     Node<E> node = validate(p);
78     return node.getRight();
79 }

```

Code Fragment 8.9: An implementation of the `LinkedBinaryTree` class.
(Continued from Code Fragment 8.8; continues in Code Fragments 8.10 and 8.11.)

```

80 // update methods supported by this class
81 /** Places element e at the root of an empty tree and returns its new Position. */
82 public Position<E> addRoot(E e) throws IllegalStateException {
83     if (!isEmpty()) throw new IllegalStateException("Tree is not empty");
84     root = createNode(e, null, null, null);
85     size = 1;
86     return root;
87 }
88
89 /** Creates a new left child of Position p storing element e; returns its Position. */
90 public Position<E> addLeft(Position<E> p, E e)
91     throws IllegalArgumentException {
92     Node<E> parent = validate(p);
93     if (parent.getLeft() != null)
94         throw new IllegalArgumentException("p already has a left child");
95     Node<E> child = createNode(e, parent, null, null);
96     parent.setLeft(child);
97     size++;
98     return child;
99 }
100
101 /** Creates a new right child of Position p storing element e; returns its Position. */
102 public Position<E> addRight(Position<E> p, E e)
103     throws IllegalArgumentException {
104     Node<E> parent = validate(p);
105     if (parent.getRight() != null)
106         throw new IllegalArgumentException("p already has a right child");
107     Node<E> child = createNode(e, parent, null, null);
108     parent.setRight(child);
109     size++;
110     return child;
111 }
112
113 /** Replaces the element at Position p with e and returns the replaced element. */
114 public E set(Position<E> p, E e) throws IllegalArgumentException {
115     Node<E> node = validate(p);
116     E temp = node.getElement();
117     node.setElement(e);
118     return temp;
119 }

```

Code Fragment 8.10: An implementation of the `LinkedBinaryTree` class.
(Continued from Code Fragments 8.8 and 8.9; continues in Code Fragment 8.11.)

```

120  /** Attaches trees t1 and t2 as left and right subtrees of external p. */
121  public void attach(Position<E> p, LinkedBinaryTree<E> t1,
122                  LinkedBinaryTree<E> t2) throws IllegalArgumentException {
123      Node<E> node = validate(p);
124      if (isInternal(p)) throw new IllegalArgumentException("p must be a leaf");
125      size += t1.size() + t2.size();
126      if (!t1.isEmpty()) { // attach t1 as left subtree of node
127          t1.root.setParent(node);
128          node.setLeft(t1.root);
129          t1.root = null;
130          t1.size = 0;
131      }
132      if (!t2.isEmpty()) { // attach t2 as right subtree of node
133          t2.root.setParent(node);
134          node.setRight(t2.root);
135          t2.root = null;
136          t2.size = 0;
137      }
138  }
139  /** Removes the node at Position p and replaces it with its child, if any. */
140  public E remove(Position<E> p) throws IllegalArgumentException {
141      Node<E> node = validate(p);
142      if (numChildren(p) == 2)
143          throw new IllegalArgumentException("p has two children");
144      Node<E> child = (node.getLeft() != null ? node.getLeft() : node.getRight());
145      if (child != null)
146          child.setParent(node.getParent()); // child's grandparent becomes its parent
147      if (node == root)
148          root = child; // child becomes root
149      else {
150          Node<E> parent = node.getParent();
151          if (node == parent.getLeft())
152              parent.setLeft(child);
153          else
154              parent.setRight(child);
155      }
156      size--;
157      E temp = node.getElement();
158      node.setElement(null); // help garbage collection
159      node.setLeft(null);
160      node.setRight(null);
161      node.setParent(null); // our convention for defunct node
162      return temp;
163  }
164  } //----- end of LinkedBinaryTree class -----

```

Code Fragment 8.11: An implementation of the `LinkedBinaryTree` class.
(Continued from Code Fragments 8.8–8.10.)

Performance of the Linked Binary Tree Implementation

To summarize the efficiencies of the linked structure representation, we analyze the running times of the `LinkedBinaryTree` methods, including derived methods that are inherited from the `AbstractTree` and `AbstractBinaryTree` classes:

- The `size` method, implemented in `LinkedBinaryTree`, uses an instance variable storing the number of nodes of a tree and therefore takes $O(1)$ time. Method `isEmpty`, inherited from `AbstractTree`, relies on a single call to `size` and thus takes $O(1)$ time.
- The accessor methods `root`, `left`, `right`, and `parent` are implemented directly in `LinkedBinaryTree` and take $O(1)$ time each. The `sibling`, `children`, and `numChildren` methods are derived in `AbstractBinaryTree` using on a constant number of calls to these other accessors, so they run in $O(1)$ time as well.
- The `isInternal` and `isExternal` methods, inherited from the `AbstractTree` class, rely on a call to `numChildren`, and thus run in $O(1)$ time as well. The `isRoot` method, also implemented in `AbstractTree`, relies on a comparison to the result of the `root` method and runs in $O(1)$ time.
- The update method, `set`, clearly runs in $O(1)$ time. More significantly, all of the methods `addRoot`, `addLeft`, `addRight`, `attach`, and `remove` run in $O(1)$ time, as each involves relinking only a constant number of parent-child relationships per operation.
- Methods `depth` and `height` were each analyzed in Section 8.1.3. The `depth` method at position p runs in $O(d_p + 1)$ time where d_p is its depth; the `height` method on the root of the tree runs in $O(n)$ time.

The overall space requirement of this data structure is $O(n)$, for a tree with n nodes, as there is an instance of the `Node` class for every node, in addition to the top-level `size` and `root` fields. Table 8.1 summarizes the performance of the linked structure implementation of a binary tree.

Method	Running Time
<code>size</code> , <code>isEmpty</code>	$O(1)$
<code>root</code> , <code>parent</code> , <code>left</code> , <code>right</code> , <code>sibling</code> , <code>children</code> , <code>numChildren</code>	$O(1)$
<code>isInternal</code> , <code>isExternal</code> , <code>isRoot</code>	$O(1)$
<code>addRoot</code> , <code>addLeft</code> , <code>addRight</code> , <code>set</code> , <code>attach</code> , <code>remove</code>	$O(1)$
<code>depth</code> (p)	$O(d_p + 1)$
<code>height</code>	$O(n)$

Table 8.1: Running times for the methods of an n -node binary tree implemented with a linked structure. The space usage is $O(n)$.

8.3.2 Array-Based Representation of a Binary Tree

An alternative representation of a binary tree T is based on a way of numbering the positions of T . For every position p of T , let $f(p)$ be the integer defined as follows.

- If p is the root of T , then $f(p) = 0$.
- If p is the left child of position q , then $f(p) = 2f(q) + 1$.
- If p is the right child of position q , then $f(p) = 2f(q) + 2$.

The numbering function f is known as a **level numbering** of the positions in a binary tree T , for it numbers the positions on each level of T in increasing order from left to right. (See Figure 8.10.) Note well that the level numbering is based on *potential* positions within a tree, not the actual shape of a specific tree, so they are not necessarily consecutive. For example, in Figure 8.10(b), there are no nodes with level numbering 13 or 14, because the node with level numbering 6 has no children.

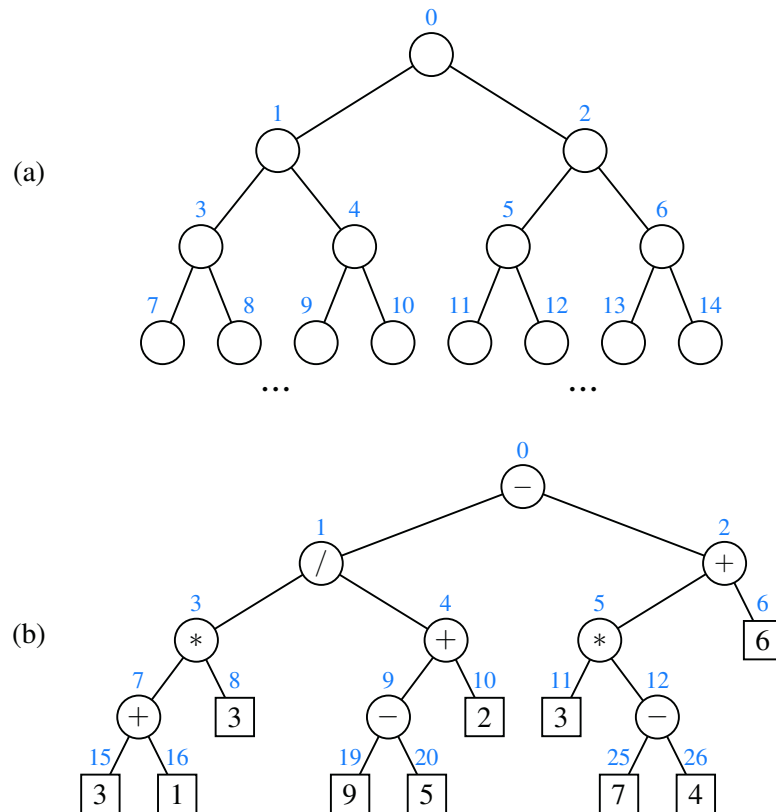


Figure 8.10: Binary tree level numbering: (a) general scheme; (b) an example.

The level numbering function f suggests a representation of a binary tree T by means of an array-based structure A , with the element at position p of T stored at index $f(p)$ of the array. We show an example of an array-based representation of a binary tree in Figure 8.11.

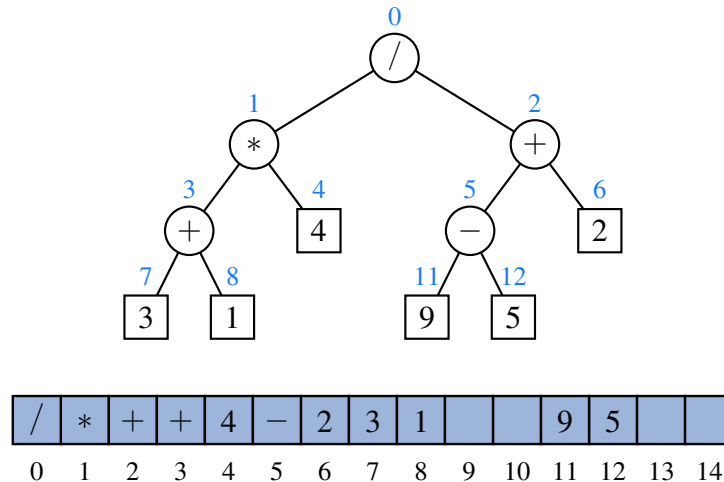


Figure 8.11: Representation of a binary tree by means of an array.

One advantage of an array-based representation of a binary tree is that a position p can be represented by the single integer $f(p)$, and that position-based methods such as `root`, `parent`, `left`, and `right` can be implemented using simple arithmetic operations on the number $f(p)$. Based on our formula for the level numbering, the left child of p has index $2f(p) + 1$, the right child of p has index $2f(p) + 2$, and the parent of p has index $\lfloor (f(p) - 1)/2 \rfloor$. We leave the details of a complete array-based implementation as Exercise R-8.16.

The space usage of an array-based representation depends greatly on the shape of the tree. Let n be the number of nodes of T , and let f_M be the maximum value of $f(p)$ over all the nodes of T . The array A requires length $N = 1 + f_M$, since elements range from $A[0]$ to $A[f_M]$. Note that A may have a number of empty cells that do not refer to existing positions of T . In fact, in the worst case, $N = 2^n - 1$, the justification of which is left as an exercise (R-8.14). In Section 9.3, we will see a class of binary trees, called “heaps” for which $N = n$. Thus, in spite of the worst-case space usage, there are applications for which the array representation of a binary tree is space efficient. Still, for general binary trees, the exponential worst-case space requirement of this representation is prohibitive.

Another drawback of an array representation is that many update operations for trees cannot be efficiently supported. For example, removing a node and promoting its child takes $O(n)$ time because it is not just the child that moves locations within the array, but all descendants of that child.

8.3.3 Linked Structure for General Trees

When representing a binary tree with a linked structure, each node explicitly maintains fields left and right as references to individual children. For a general tree, there is no a priori limit on the number of children that a node may have. A natural way to realize a general tree T as a linked structure is to have each node store a single *container* of references to its children. For example, a children field of a node can be an array or list of references to the children of the node (if any). Such a linked representation is schematically illustrated in Figure 8.12.

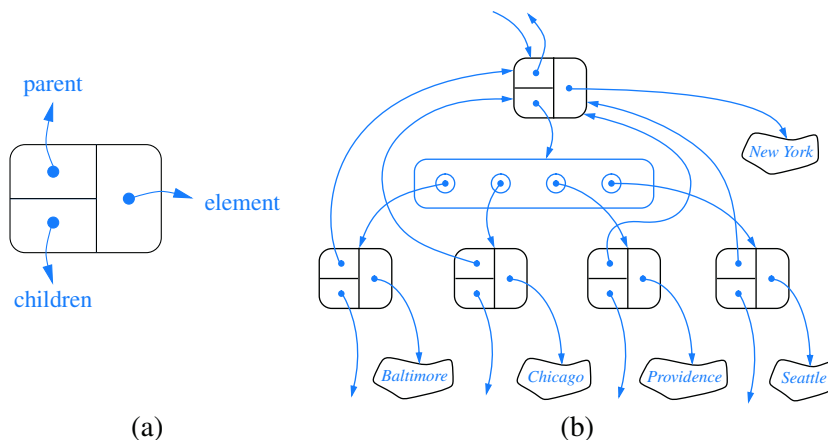


Figure 8.12: The linked structure for a general tree: (a) the structure of a node; (b) a larger portion of the data structure associated with a node and its children.

Table 8.2 summarizes the performance of the implementation of a general tree using a linked structure. The analysis is left as an exercise (R-8.13), but we note that, by using a collection to store the children of each position p , we can implement $\text{children}(p)$ by simply iterating that collection.

Method	Running Time
size, isEmpty	$O(1)$
root, parent, isRoot, isInternal, isExternal	$O(1)$
$\text{numChildren}(p)$	$O(1)$
$\text{children}(p)$	$O(c_p + 1)$
$\text{depth}(p)$	$O(d_p + 1)$
height	$O(n)$

Table 8.2: Running times of the accessor methods of an n -node general tree implemented with a linked structure. We let c_p denote the number of children of a position p , and d_p its depth. The space usage is $O(n)$.